From Ratings to Recommendations: Matrix Factorization with TensorFlow.js

HW5 — LLM4Rec Course, HSE University

Student: *[insert name before submission]*

Instructor: Seungmin Jin (sedzhin@hse.ru)

Date: October 13, 2026

Abstract

This work addresses sparse rating-based recommendation by factorizing the MovieLens 100K user–movie rating matrix with 16-dimensional latent factors trained in TensorFlow.js. Ratings are split per user in strictly chronological order into 80 % train, 10 % validation and 10 % test partitions. The model predicts the global training mean plus a latent dot product, is trained with mini-batch Adam and L2 regularization, and is checkpointed on validation RMSE with restoration of the actual best-epoch weights. The factorization attains a test RMSE of **1.0611** against a training-mean baseline of **1.2289** (13.7 % reduction) and an (uncut) Recall@10 of **0.0098** over 861 evaluable users, with evaluation exclusions reported explicitly. A two-user analysis of the Top-10 lists shows how the model balances familiar, taste-consistent titles against long-tail discovery, and what that implies for a streaming platform.

---

1. Problem Statement

Given a sparse log of 100 000 explicit ratings, learn user and movie representations that (i) predict unseen ratings accurately and (ii) retrieve relevant movies a user has not yet seen. Unobserved entries of the rating matrix are **missing data, not zeros**: the implementation never materializes a dense matrix of observed-vs-zero targets; every loss term and every metric is computed over observed ratings only.

All six required functions (`predictRating`, `initializeFactors`, `trainMF`, `selectBestCheckpoint`, `recommendTopK`, `recallAtK`) are implemented in `week5/script.js` with TensorFlow.js tensor operations (`tf.variable`, `tf.randomNormal`, `tf.gather`, `tf.matMul`, `tf.losses.meanSquaredError`, `tf.train.adam`, `tf.topk`, `tf.tidy`); the provided scaffolding (`data.js`, `index.html`, `style.css`, renderers, metric helpers, test harness) is unmodified.

2. Dataset

MovieLens 100K [1], [2]: 943 users, 1 682 movies, 100 000 ratings on a 1–5 scale with Unix timestamps. The model's factor tensors carry a sentinel row/column 0 so that tensor row index equals the raw id; the **conceptual** rating matrix is 943 x 1 682 = 1 586 126 cells.

Quantity	Value (measured)
Users / movies / ratings	943 / 1 682 / 100 000
Conceptual matrix	943 x 1 682 = 1 586 126 cells
Density (observed / cells)	100 000 / 1 586 126 = **6.305 %**
Sparsity (1 - density)	**93.695 %**

3. Methodology

3.1 Chronological split

Each user's ratings are sorted by ascending timestamp (ties broken by movie id) and cut into train = first `floor(0.8n)`, validation = next `floor(0.1n)`, test = the remainder, mimicking "predict the future from the past". Users with fewer than 5 ratings are excluded from splitting; MovieLens 100K contains none, so **0 users were skipped**.

Split	Ratings	Share
Train	79 619	79.619 %
Validation	9 596	9.596 %
Test	10 785	10.785 %

3.2 Model

Following the matrix-factorization formulation of Koren, Bell and Volinsky [3], the model stores the global **training** mean mu and 16-dimensional vectors `p_u` (user) and `q_i` (movie):

> *r_hat(u, i) = mu + p_u . q_i*

clamped to [1, 5] at prediction time. The factor matrices are dense tensors of shape [944, 16] and [1683, 16] indexed by raw ids, initialized with `tf.randomNormal` (mean 0, standard deviation 0.05). mu = **3.580603** is computed over the train split only; validation and test ratings never enter mu, the loss, or any gradient.

3.3 Training

Adam [4] (learning rate 0.01) minimizes, over mini-batches of 64 shuffled training ratings, the mean squared error of unclamped predictions plus L2 regularization of the gathered factor rows, scaled by 1/B so the per-rating penalty is reg * (||p_u||^2 + ||q_i||^2) regardless of batch size (reg = 0.05). All batch work is expressed as tensor operations (`tf.gather` of the batch rows, one `tf.matMul`-equivalent dot product via element-wise multiply and sum); per-step intermediates live inside `tf.tidy` / `optimizer.minimize`, batch index tensors are explicitly disposed, and the optimizer object is disposed after the loop. Training runs for 20 epochs; train and validation RMSE are recomputed after every epoch with a single batched tensor pass each.

3.4 Checkpoint selection

The checkpoint is the epoch with the smallest validation RMSE (ties resolved toward the earliest epoch). The implementation snapshots the weight **values** of every improving epoch (`dataSync` copies) and, after

the loop, re-assigns the best snapshot into the live variables, so the fitted model genuinely holds the best-epoch weights rather than the last-epoch weights. On the reported run the best validation RMSE (1.0126, epoch 2) differs from the last epoch (1.0317, overfitted), and the restored model reproduces 1.0126 exactly.

4. Evaluation Methodology

Rating prediction. RMSE of clamped predictions over a split. The baseline predicts the train mean mu for every pair (provided `computeBaselineRMSE`); its value was independently recomputed from a raw parse of `u.data` and matched exactly.

Retrieval. Recall@10 treats test ratings >= 4.0 as relevant. The candidate pool is the set of movie ids appearing anywhere in the **train** split (global, not per-user). Each user's Top-10 list excludes every movie the user rated in train or validation, so a relevant test movie already rated in training cannot be recalled; such movies are counted in the exclusion report. Per-user recall = hits / relevant candidate movies; Recall@10 is the mean over evaluable users (users in the test split with at least one relevant candidate movie). Recommendation scores shown are the raw, unclamped values mu + p_u . q_i, used for ranking.

5. Results

All figures below are from one measured run of `week5/tests/run-real-pipeline.js` with the assignment defaults (20 epochs, lr 0.01, reg 0.05, batch 64, 16 factors) using `@tensorflow/tfjs-node` 4.22.0 (native TensorFlow backend). An independent re-run through the page's own code path (bootstrap, button handlers, renderers — `week5/tests/ui-flow.test.js`, pure-JavaScript `cpu` backend) reproduced the same conclusions.

Metric	Value
Train mean mu	3.580603
Baseline validation RMSE	1.1830
Baseline test RMSE	1.2289
Best validation RMSE	**1.0126 (epoch 2)**
Restored-model validation RMSE	1.0126 (matches best epoch)
MF test RMSE	**1.0611**
Improvement vs. baseline test RMSE	-0.1678 (-13.7 %)
Recall@10	**0.0098** over 861 evaluable users
Avg. relevant candidates / user	5.972
Avg. hits / user	0.078
Excluded: unseen relevant test items	13 (ids 913, 1106, 1130, 1458, 1498, 1525, 1613, 1625, 1645, 1646, 1650, 1651, 1674)
Excluded: users without a relevant candidate	82 (of 943)

Training curve (train / validation RMSE per epoch; best at epoch 2):

Epoch	Train RMSE	Val RMSE		Epoch	Train RMSE	Val RMSE
1	0.8845	1.0297		11	0.7116	1.0276
2	0.8034	**1.0126**		12	0.7088	1.0312
3	0.7630	1.0163		13	0.7076	1.0346
4	0.7445	1.0215		14	0.7090	1.0392
5	0.7341	1.0275		15	0.7080	1.0403
6	0.7254	1.0277		16	0.7078	1.0339
7	0.7214	1.0285		17	0.7076	1.0335
8	0.7163	1.0358		18	0.7066	1.0335
9	0.7141	1.0283		19	0.7056	1.0351
10	0.7120	1.0339		20	0.7032	1.0317

The factorization **beats the training-mean baseline** on the held-out test split by a wide margin, while validation RMSE degrading after epoch 2 demonstrates concretely why the val-based checkpoint is required. Recall@10 is low in absolute terms — Top-10 is drawn from a pool of roughly 1 600 movies against on average 5.97 relevant items per user, with the model deliberately stopped before memorization — and the exclusion counts (13 unseen relevant items, 82 users without a relevant candidate, leaving 861 evaluable users) are reported so the number is interpretable.

6. Analysis: Familiar Choices vs. New Discoveries

Both lists below come from the measured run above; neither user's list contains any movie they rated in training or validation (verified programmatically, intersection = 0).

User 196 rated 34 movies in train+validation, average **3.68** (an easy grader). Genre profile: Comedy (27), Drama (11), Romance (10). He gave 5 stars to *Babe* (1995), *Ace Ventura: Pet Detective* (1994), *A Fish Called Wanda* (1988), *Secrets & Lies* (1996), *The English Patient* (1996), *Stand by Me* (1986), *Being There* (1979) and *The American President* (1995) — a comedy-forward profile with prestige-drama accents. His Top-10:

#	Score	Title	Genres
1	4.5077	The Magic Hour (1998)	Drama
2	4.2554	The Saint of Fort Washington (1993)	Drama
3	4.2437	A Perfect World (1993)	Action, Drama
4	4.2186	Top Hat (1935)	Comedy, Musical, Romance
5	4.1926	Steel (1997)	Action
6	4.1499	Ruby in Paradise (1993)	Drama
7	4.1438	Manhattan (1979)	Comedy, Drama, Romance
8	4.1383	A Little Princess (1939)	Children's, Drama
9	4.1102	The Stupids (1996)	Comedy
10	4.1060	Selena (1997)	Drama, Musical

Familiar, taste-consistent choices: **Top Hat**, **Manhattan**, **The Stupids** and **Selena** are Comedy/Musical/Romance titles that sit squarely inside his most-rated genres, and the Drama entries (*The Saint of Fort Washington*, *Ruby in Paradise*, *A Little Princess*) echo the prestige-drama side of his 5-star set (*Secrets & Lies*, *The English Patient*). *Discovery:* every title is unseen by him, but the list reaches well beyond his rating history — 1930s–40s catalogue titles (*Top Hat*, *A Little Princess*, *Manhattan*) and little-known 1990s releases (*The Magic Hour*, *Steel*, *Ruby in Paradise*) that a popularity ranker would never surface.

User 405 rated 662 movies, average **1.92** (a harsh, heavy rater). Genre profile: Drama (257), Comedy (215), Action (118), Romance (97), Thriller (93). His 5-star set is a hall of mainstream favourites: *The Usual Suspects*, *Braveheart*, *Taxi Driver*, *The Net*, *Ed Wood*, *Star Wars*, *The Shawshank Redemption*, *While You Were Sleeping*. His Top-10:

#	Score	Title	Genres
1	5.3785	La Fille seule / A Single Girl (1995)	Drama
2	5.2060	Radioland Murders (1994)	Comedy, Mystery, Romance
3	5.1202	Hearts and Minds (1996)	Drama
4	5.0208	The Turning (1992)	Drama
5	4.8718	Aiqing wansui (1994)	Drama
6	4.7446	Boy's Life 2 (1997)	Drama
7	4.7073	New York Cop (1996)	Action, Crime
8	4.6998	Wallace & Gromit: The Best of Aardman Animation (1996)	Animation
9	4.6342	Faster Pussycat! Kill! Kill! (1965)	Action, Comedy, Drama
10	4.5860	Window to Paris (1994)	Comedy

Familiar, taste-consistent choices: the genre mix (Drama x5, Comedy x3, Action x2) is exactly his profile, and *Wallace & Gromit* plausibly matches his Comedy + Children's affinity. *Discovery:* unlike user 196, almost none of the titles resemble his 5-star blockbusters; the list is dominated by long-tail, international and cult releases (*La Fille seule*, *Aiqing wansui*, *Window to Paris*, *Faster Pussycat!*). Scores above 5.0 are raw, unclamped ranking scores mu + p_u . q_i: a user who rates almost everything 1–2 acquires large-magnitude factors, which pushes even moderately scored items to extreme values — a signal the recommender would be wise to recalibrate (e.g., per-user bias terms) before displaying scores to users.

Comparison and business value. The two users receive visibly different treatment. The moderate, comedy-loving user 196 gets a **familiarity-leaning** list: genre-faithful titles, several of them classics with broad appeal (*Top Hat*, *Manhattan*), where the discovery component is catalogue depth rather than genre risk. The harsh, high-volume user 405 gets a **discovery-leaning** list: genre-consistent but title-wise far from his favourites, surfacing library depth that a popularity baseline would hide. For a streaming platform, familiarity is the retention lever — a first slot of safe, taste-consistent picks maximizes click-through and trust — while discovery fights filter bubbles, monetizes the long tail of the catalogue, and is what a heavy rater (who has already seen the popular catalogue) actually needs. The measured lists show the factorization naturally interpolates between the two regimes; the platform should keep the familiar head of the list stable and use the tail for exploration, and should recalibrate extreme users before trusting raw scores.

7. Verification and Reproducibility

Every number in this report was produced by executing the repository code; no value was estimated by hand. Evidence:

1. **Built-in harness** (in-page "Run tests"): **11 passed, 0 failed, 0 pending** — both in the browser page flow (driven headlessly) and headlessly in Node.
2. **Independent regression suite** (`week5/tests/regression.test.js`): **18/18 checks pass**, covering unknown/NaN ids, empty datasets, clamping, factor shapes, best-epoch weight restoration (constructed diverging run where the model matched the best, not the last, epoch), train/val exclusion, the Recall@10 denominator and global candidate pool, `tf.memory().numTensors` leak checks, and repeated training.
3. **Application-flow suite** (`week5/tests/ui-flow.test.js`): **all checks pass** by executing the real `window.onload` bootstrap and the real Train / Recommend / Recall / Run-tests handlers with page defaults; the restored model's validation RMSE equals the best epoch's, and Test RMSE 1.0467 < baseline 1.2289 in that independent re-run.
4. **Hand cross-check**: `predictRating(196, 50)`, `(405, 100)`, `(1, 1)` match an independent plain-JavaScript dot product over the raw factor arrays to ~1e-8; the baseline RMSE matches an independent raw parse of `u.data` exactly.
5. **Tensor accounting**: after training exactly the 2 model variables remain alive (initializer tensors, batch tensors, Adam slots and checkpoint-snapshot tensors all disposed); after final disposal 0 tensors remain.

Reproduction: `cd week5/tests && npm install && npm test && npm run ui-flow && npm run real-pipeline`; the page runs via `python3 -m http.server` in `week5/` with the TensorFlow.js 4.22.0 CDN build (WebGL backend in a browser). Run-to-run variation (best validation RMSE 1.0009–1.0192, test RMSE 1.0419–1.0611, Recall@10 0.0069–0.0155 across the runs performed) stems from the unseeded weight initialization and shuffled batches; the split, mu, baselines and exclusion counts are identical in every run. **Not verified:** execution on the WebGL (GPU) backend — the desktop browser was unavailable during final validation; all JavaScript code paths of the page were exercised headlessly instead, and `initTf` selects WebGL automatically in a browser.

8. Conclusion

The 16-dimensional factorization trained with TensorFlow.js reduces held-out test RMSE from 1.2289 (train-mean baseline) to 1.0611 and produces Top-10 lists that demonstrably exclude train/validation history, balance familiar taste-consistent titles with genuine long-tail discovery, and are accompanied by a fully interpretable retrieval metric (Recall@10 with explicit exclusions). Validation-based checkpointing with true weight restoration is not cosmetic: on the reported run it is the difference between a 1.0126 and a 1.0317 validation model.

AI Usage Declaration

The six required functions, the standalone test suites (`week5/tests/`), the headless validation of the page's code paths, and a draft of this report were produced with an AI coding assistant (OpenCode, session `ses_ecaf12cb3ffeUdR1P2GEELQTRL`) under student supervision. Every quantitative claim was obtained by executing the code in this repository; the exact test logs and pipeline outputs are reproducible with the commands in Section 7. Citations were limited to sources verified in the assignment readme (MovieLens

100K [1], [2]) and established literature (matrix factorization [3], Adam [4]); the TensorFlow.js library is cited as the software actually loaded by the page [5]. Browser-based (WebGL) visual verification was **not** performed and is not claimed; see Section 7.

References

[1] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," *ACM Transactions on Interactive Intelligent Systems*, 2015. (As cited in `week5/readme.md`, Section 3.)

[2] GroupLens, "MovieLens 100K Dataset," https://grouplens.org/datasets/movielens/100k/ (dataset page cited in `week5/readme.md`, Section 3).

[3] Y. Koren, R. Bell and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.

[4] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," arXiv:1412.6980, 2014.

[5] TensorFlow.js, version 4.22.0 (the browser build loaded by `week5/index.html` from `https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@4.22.0/dist/tf.min.js`), https://www.tensorflow.org/js.